creating customized indexes tailored for production scenarios. The chapter also looks at working with index templates, discussing the mechanics of index and composable templates. The final section investigates index lifecycle management.

- Chapter 7 immerses us in text analysis, examining how full text is tokenized and normalized using Elasticsearch's analyzer modules. We look at the mechanics of text analysis, exploring built-in analyzers like the `standard`, `simple`, `keyword`, and language analyzers. This chapter will empower you with the knowledge to create custom analyzers.

Chapters 8 through 13 are dedicated to search:

- Chapter 8 lays the groundwork for understanding the fundamentals of search, explaining the mechanics of how search requests are processed and responses generated. We introduce two primary types of search: URL request and Query DSL. We also examine cross-cutting features like highlighting, sorting, pagination, and others, providing a comprehensive introduction to Elasticsearch's search functionality.
- Chapter 9 explores the realm of term-level queries, which are specifically designed for structured data. The chapter offers a detailed examination of various types of term-level queries, including range, prefix, wildcard, and fuzzy queries.
- Chapter 10 looks at full-text queries designed specifically for searching unstructured data. The chapter examines the mechanics of using full-text search APIs, employing a variety of queries including match family queries, query strings, fuzzy queries, and simple string queries, among others.
- Chapter 11 navigates the intricate world of compound queries, highlighting the Boolean query as a versatile tool for crafting advanced search queries. We look at the use of conditional clauses like `must`, `must_not`, `should`, and `filter` to structure leaf queries into more complex, compound queries. The chapter concludes with a detailed examination of boosting and `constant_score` queries.
- Chapter 12 introduces specialized queries, including `distance_feature`, `percolator`, `more_like_this`, and `pinned`. The chapter examines the unique benefits of each type of query, such as the `distance_feature` query's ability to prioritize results closer to a given location and the `more_like_this` query's function of finding similar-looking documents. The `percolator` query, which alerts users to newly available results, is also examined in detail.
- Chapter 13 offers a detailed examination of aggregations. We explore metrics aggregations, generating statistics like sum, average, minimum, maximum, top hits, and mode. The chapter also highlights the use of bucket aggregations in collecting aggregated data into sets of buckets. We also look at pipeline aggregations, which provide advanced statistical analytics like derivatives and moving averages.